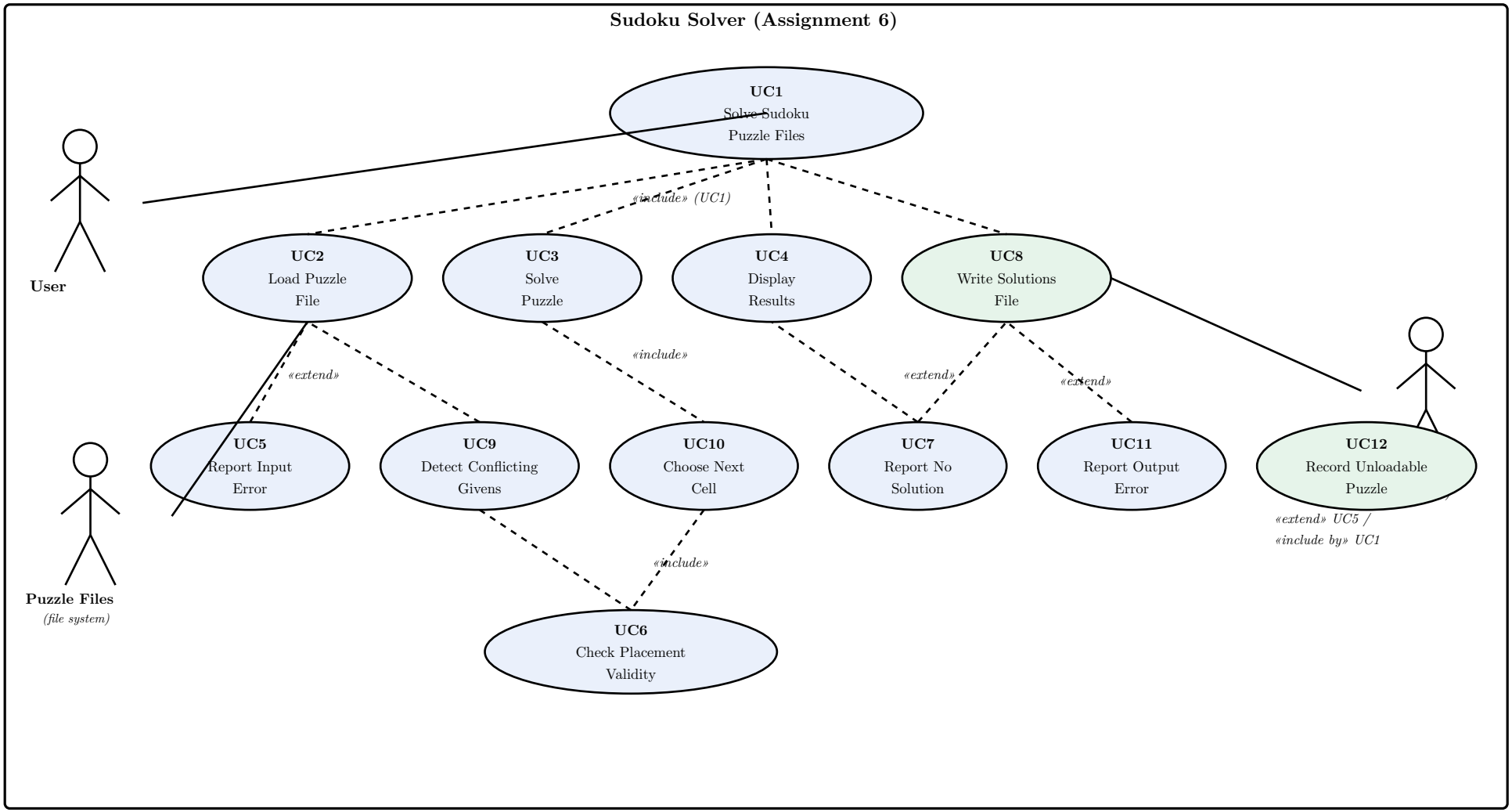
EECS 348 Assignment 6 – UML Use Case Diagram

Sudoku solver, revised from Jacob Fox's Assignment 5 (itself revised from Lucas Frias's Assignment 4). Updated for Assignment 6 with a GenAI assistant (Claude Opus 5.5), then checked against Assignment6.cpp.



Use case carried over from Assignment 5

New or structurally changed in Assignment 6

— association ···· «include» / «extend»

Actors. Primary: **User** (starts the program from a terminal). Secondary: **Puzzle Files** (puzzle1.txt-puzzle5.txt, read by the program) and **Solutions File** (solutions.txt – in Assignment 5 this was five separate files, solution1.txt-solution5.txt; Assignment 6 merges them into one shared file written by every puzzle in turn).

Relationships. UC1 includes UC2, UC3, UC4 and UC8. UC3 includes UC10. UC9 and UC10 include UC6. UC5 and UC9 extend UC2. UC7 extends UC4 and UC8. UC11 extends UC1 (moved from UC8, since the file is now opened/closed once for the whole run instead of once per puzzle). **UC12 (new) extends UC5 and is included by UC1** as the alternate path taken whenever a puzzle fails to load.

2. Identified Use Cases

ID	Use case	Relationship	Status	Brief description
UC1	Solve Sudoku Puzzle Files	Includes UC2, UC3, UC4 and UC8	Updated	User runs the program; one shared solutions.txt is opened once; every puzzle file is loaded and either solved, displayed and written, or (if unloadable) has its failure recorded directly in the shared file.
UC2	Load Puzzle File	Included by UC1; extended by UC5, UC9	Unchanged	Read 81 cells from one file into the 9x9 grid and the digit tables; refuse files with extra data.
UC3	Solve Puzzle	Included by UC1; includes UC10	Unchanged	Search for every valid completion by recursion and backtracking, always branching on the most constrained cell.
UC4	Display Results	Included by UC1; extended by UC7	Updated	Print file name and starting puzzle on the console, then the final solution count (or run UC7); no longer prints every individual solution grid to the console – only to the shared file (UC8).
UC5	Report Input Error	Extends UC2; now also feeds UC12	Unchanged logic	Print an error and abandon this puzzle when it cannot be opened or parsed; the failure now also reaches UC12 so the shared file still gets a section for it.
UC6	Check Placement Validity	Included by UC9, UC10	Unchanged	Work out which digits may go into a cell under the row, column and block rules, with three table lookups.
UC7	Report No Solution	Extends UC4 and UC8	Unchanged logic	Report “No solution found” on the console and write “No solution found.” as this puzzle’s section of the shared file.
UC8	Write Solutions File	Included by UC1; extended by UC7	Updated	Write this puzzle’s section – a “Puzzle: x” header, then every solution (file only) and the count, or “No solution found.” – into the one solutions.txt that main() opened once and every solver writes into by reference.
UC9	Detect Conflicting Givens	Extends UC2; includes UC6	Unchanged	Notice when two given digits break a Sudoku rule so an impossible puzzle is not reported as solved.
UC10	Choose Next Cell	Included by UC3; includes UC6	Unchanged	Pick the empty cell with the fewest legal digits as the next cell to branch on.
UC11	Report Output Error	Extends UC1	Updated	Print an error if the shared solutions.txt cannot be opened at the start, or was not fully written when closed at the end – checked once for the whole run instead of once per puzzle.
UC12	Record Unloadable Puzzle	Extends UC5; included by UC1	**New**	When a puzzle cannot be loaded, still write a “Puzzle: x” / “No solution found.” section for it into the shared file, so every listed puzzle appears in solutions.txt even if none was ever solved.

What changed in the model (Assignment 5 to Assignment 6)

Item	Assignment 5	Assignment 6
Actors	User, Puzzle Files, Solution Files (5 files)	User, Puzzle Files, Solutions File (1 shared file)
Use cases	11 (UC1-UC11)	12: UC1, UC4, UC5, UC7, UC8, UC11 updated; UC12 new; UC2/3/6/9/10 unchanged
Relationships	UC11 extends UC8	UC11 extends UC1 (the file now opens/closes once for the run, not once per puzzle); new: UC12 extends UC5, included by UC1
Where answers go	One file per puzzle, solutionN.txt for puzzleN.txt	One shared file, solutions.txt, with a “Puzzle: x” header starting each puzzle’s section
Console output (UC4)	Every solution grid printed to console **and** file	Solution grids go to the file only; console shows the file name, starting puzzle and final count

Item	Assignment 5	Assignment 6
Unloadable puzzle	Silently skipped – no solutionN.txt produced for it at all	Still gets a “Puzzle: x” / “No solution found.” section in solutions.txt (UC12)
solver’s file handle	Each SudokuSolver opens and owns its own std::ofstream	main() opens one std::ofstream and every SudokuSolver holds a std::ostream& reference to it

3. Use Case Descriptions – Updated and New Use Cases

UC2, UC3, UC6, UC9 and UC10 are unchanged from the Assignment 5 model (Assignment5UseCase.pdf, Section 3) and are not repeated here; only the use cases affected by the single shared output file are described in full below.

Use case UC1: Solve Sudoku Puzzle Files	[UPDATED IN A6]
Actors	User (primary); Puzzle Files (secondary); Solutions File (secondary)
Relationships	Includes UC2, UC3, UC4 and UC8; the failure path includes UC12
Brief description	The user runs the program. One shared solutions.txt is opened once, then puzzle1.txt through puzzle5.txt are processed in order against it.
Preconditions	Program is compiled and run from a folder that holds puzzle1.txt-puzzle5.txt and lets the program create solutions.txt.
Step-by-step flow	1. User runs the executable with no arguments. 2. System builds the list of five puzzle file names. 3. System opens solutions.txt once, before any puzzle is processed; if this fails, UC11 reports it and the program stops (return 1). 4. For each file name, system creates one solver object that holds a reference to the already-open file (no per-puzzle file name is built any more). 5. System loads the puzzle (UC2). If loading fails, system records the failure in solutions.txt (UC12) and moves to the next file. 6. If loading succeeds, system solves the puzzle (UC3), displays results on the console (UC4) and writes this puzzle's section to solutions.txt (UC8). 7. After the fifth file, system closes solutions.txt, lets UC11 confirm the write succeeded, and exits with code 0 (or 1 if the file could not be opened or saved).
Alternate flows	A file that fails to load does not stop the run; UC12 still gives it a section in solutions.txt and the remaining files are processed. If solutions.txt cannot be created at all, the run stops after reporting the error (new in A6 – in A5 this could only fail one puzzle at a time).
Postconditions	solutions.txt holds one section per puzzle file (solved, unsolved, or unloadable). Program has exited with code 0, or 1 if the shared file could not be opened or saved.
Detailed description	The main routine still holds the five file names and loops over them, but it no longer builds a per-puzzle output file name with a counter: it opens solutions.txt exactly once, and every solver object it creates receives that same stream by reference (std::ostream&), so nothing about the puzzle loop changes except where the answers end up. Each pass still creates one fresh solver object, so no search state carries over between puzzles – only the output stream is shared.
Implemented by	main()
Changed from A5	Removes the per-puzzle counter and solutionN.txt name; opens one shared ofstream before the loop instead of inside it; adds the UC12 failure path so every puzzle, loadable or not, gets a section in the output file; moves the open/close error checks (UC11) out to this use case.

Use case UC4: Display Results	[UPDATED IN A6]
Actors	User (reads output)
Relationships	Included by UC1; extended by UC7
Brief description	Print the file name and the starting puzzle to the console, then either the final solution count or "No solution found" – individual solution grids are no longer echoed to the console, only written to solutions.txt (UC8).
Preconditions	Solver has started on a loaded puzzle.
Step-by-step flow	1. Print a separator line and "File: <name>". 2. Print the starting puzzle with "_" for empty cells. 3. (UC3 finds each solution and UC8 writes it to the file directly; this use case is not re-entered per solution any more.) 4. When the search ends, print "Solutions found: N", or run UC7 if N is zero. 5. Print a closing separator line.

Use case UC4: Display Results	[UPDATED IN A6]
Alternate flows	If the givens clash, print "Two of the given digits clash, so this puzzle has no solution." before UC7 runs.
Postconditions	Terminal shows the file name, starting puzzle and final outcome for that puzzle, but not the solution grids themselves.
Detailed description	In Assignment 5, recordSolution() printed every solution to std::cout as well as to the solution file – for a puzzle with many solutions this meant every grid was formatted and written twice. Assignment6GenAI.pdf's benchmarking showed this roughly doubled the per-solution cost of the search for puzzles with large search spaces. Removing the console copy keeps correctness and the file's content identical while measurably speeding up the program; the console now summarizes each puzzle instead of fully re-displaying it.
Implemented by	SudokuSolver::solve(), printGrid(std::ostream&)
Changed from A5	recordSolution() no longer writes to std::cout; the "Saved to solutionN.txt" line is removed from here (the save confirmation now happens once for the whole run, in UC11).

Use case UC5: Report Input Error	[LOGIC UNCHANGED; NEW RELATIONSHIP IN A6]
Actors	User (reads output)
Relationships	Extends UC2; its failure now also drives UC12 (new in A6)
Brief description	Print a clear error for a bad puzzle file to the console and abandon loading it; the caller (UC1) now also runs UC12 so the shared file still records this puzzle.
Preconditions	UC2 hit a problem (file missing, too short, bad character, or trailing data).
Step-by-step flow	1. Identify the problem. 2. Print "Error: <reason>" with the file name to the error stream. 3. Return failure to UC1, which runs UC12 instead of UC3/UC4/UC8 for this file.
Alternate flows	None.
Postconditions	No solve is attempted for this file; solutions.txt still receives a "No solution found." section for it, via UC12.
Detailed description	The four error conditions themselves (cannot open, too few cells, bad character, extra data) are identical to Assignment 5 – loadPuzzle() was not touched by the single-file change. What changed is what happens to the overall run afterward: in Assignment 5 a failed load simply meant no solutionN.txt existed for that puzzle; in Assignment 6 every puzzle shares one file, so UC1 now always calls either UC8 (on success) or UC12 (on this failure) so solutions.txt never has a silent gap.
Implemented by	SudokuSolver::loadPuzzle() error branches
Changed from A5	No change inside loadPuzzle() itself; the caller's response to a UC5 failure changed (see UC12).

Use case UC7: Report No Solution	[LOGIC UNCHANGED IN A6]
Actors	User (reads output); Solutions File (secondary)
Relationships	Extends UC4 and UC8
Brief description	Tell the user, on screen and in the shared file, that no completion exists for a puzzle that was actually loaded and searched (as opposed to one that failed to load – see UC12).
Preconditions	The search finished, or was skipped because the givens clash, with a solution count of zero.
Step-by-step flow	1. Check that the count is zero. 2. Console: print "No solution found". 3. File: write the single line "No solution found." as this puzzle's section of solutions.txt. 4. Continue with the closing steps of UC4 and UC8.
Alternate flows	None.
Postconditions	This puzzle's section of solutions.txt is exactly the one line.
Detailed description	Unchanged from Assignment 5: a puzzle has no solution either because two givens clash directly (UC9) or because the search exhausts every branch (puzzle4.txt in the test set). The only difference from Assignment 5 is that this one line now lands inside a shared file's section rather than being the entire content of its own solutionN.txt.

Use case UC7: Report No Solution	[LOGIC UNCHANGED IN A6]
Implemented by	SudokuSolver::solve() (the solutionsFound == 0 branch)
Changed from A5	None in logic; only the destination (a section of solutions.txt instead of a standalone file) changed.

Use case UC8: Write Solutions File	[UPDATED IN A6 (renamed from Write Solution File)]
Actors	Solutions File (secondary)
Relationships	Included by UC1; extended by UC7
Brief description	Write this puzzle's section – a "Puzzle: x" header, then either every solution grid and the solution count, or "No solution found." – into the one solutions.txt that every puzzle shares.
Preconditions	The puzzle loaded successfully; solutions.txt is already open (UC1/UC11).
Step-by-step flow	1. Write the header line "Puzzle: <filename>" (new in A6 – needed so a shared file can still be split back into per-puzzle sections). 2. As UC3 finds each solution, write its nine rows in the puzzle-file layout, then one blank line (file only – see UC4). 3. When the search ends, write "Solutions found: n" as the next line, or let UC7 write "No solution found." instead. 4. Write one further blank line to separate this puzzle's section from the next puzzle's.
Alternate flows	No solutions: UC7 writes only "No solution found." instead of grids and a count.
Postconditions	solutions.txt's section for this puzzle is complete; the file itself stays open for the next puzzle (it is not closed here any more – see UC11).
Detailed description	In Assignment 5 every SudokuSolver owned its own std::ofstream and opened, wrote and closed one solutionN.txt per puzzle. In Assignment 6 the solver instead holds a std::ostream& reference to a single std::ofstream that main() opens once before the loop and closes once after it; this use case's job narrows to writing one well-delimited section into that shared stream rather than managing a file's whole lifecycle. The "Puzzle: x" header is the only genuinely new output text this use case introduces.
Implemented by	SudokuSolver::solve() (header line, summary line), recordSolution(), printGrid(std::ostream&)
Changed from A5	No longer opens/creates/closes a file (moved to main()/UC1/UC11); adds the "Puzzle: x" header line; drops the per-file "Saved to" confirmation (moved to UC11, once for the whole run).

Use case UC11: Report Output Error	[UPDATED IN A6]
Actors	User (reads output)
Relationships	Extends UC1 (moved from UC8)
Brief description	Tell the user if the single shared solutions.txt could not be opened at the very start of the run, or was not fully written when it was closed at the very end.
Preconditions	main() has either just tried to open solutions.txt (start of run) or just closed it (end of run).
Step-by-step flow	1. At the start: if solutions.txt could not be opened, print "Error: Could not create output file solutions.txt" and stop the program immediately (return 1) – no puzzles are processed at all. 2. At the end: after the loop, close solutions.txt and test the stream state. 3. If the close failed, print "Error: Could not write solutions.txt" (return 1); otherwise print "Saved to solutions.txt" (return 0).
Alternate flows	None.
Postconditions	The user knows whether solutions.txt was saved for the whole run.
Detailed description	Assignment 5 checked this once per puzzle, right after each solutionN.txt was closed, so one puzzle's save failure never affected any other puzzle's file. Assignment 6 trades that per-puzzle isolation for a single shared file: a failure to even open solutions.txt now stops the entire run before any puzzle is attempted, and a failure to flush it at the end is reported once for every puzzle at once, since by then every puzzle's section already lives in the same stream.
Implemented by	main() (the two error checks around solutionsFile)

Use case UC11: Report Output Error	[UPDATED IN A6]
Changed from A5	Moved from SudokuSolver::solve() (once per puzzle) to main() (once per run, at the start and at the end); an open failure is now fatal to the whole run rather than to one puzzle.

Use case UC12: Record Unloadable Puzzle	[NEW IN A6]
Actors	Solutions File (secondary)
Relationships	Extends UC5; included by UC1 as the alternative to UC3/UC4/UC8
Brief description	When a puzzle cannot be loaded, still write a section for it in solutions.txt – the header line and "No solution found." – so every one of the five puzzles appears in the shared file, even the ones that were never actually solved.
Preconditions	UC2 has just returned failure (via UC5) for this puzzle.
Step-by-step flow	1. Write "Puzzle: <filename>" to the shared file. 2. Write "No solution found." followed by a blank line. 3. Return to UC1, which moves on to the next puzzle file.
Alternate flows	None.
Postconditions	solutions.txt has a complete, well-formed section for this puzzle even though it was never solved or even fully read.
Detailed description	This use case exists purely because of the move to a single shared output file. In Assignment 5, a puzzle that failed to load simply produced no solutionN.txt – the **absence** of a file was itself the signal. That signal disappears once every puzzle writes into the same solutions.txt: silently skipping a puzzle would leave a gap in the file indistinguishable from a bug. UC12 closes that gap by giving a failed-to-load puzzle the exact same two-line "no solution" section that a puzzle UC7 proves unsolvable gets, just reached from a different trigger (a load failure rather than an exhausted search).
Implemented by	SudokuSolver::recordLoadFailure()
Changed from A5	New use case, required entirely by the switch to a single shared output file.